Continuous Control I: Deterministic Policy Gradients & DDPG

Naeemullah Khan
naeemullah.khan@kaust.edu.sa

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

August 9, 2026

Table of Contents

Continuous Control: **Recap (Day 4)**

Recap: What Day 4 Established

- **Actor-Critic**: an actor $\pi_\theta(a|s)$ updated by the policy gradient, plus a critic $V_\phi(s)$ trained by TD learning to supply a low-variance advantage estimate $\hat{A}_t$:

$$\nabla_\theta \mathcal{J}(\theta) = \mathbb{E}_t\left[\nabla_\theta \log \pi_\theta(a_t|s_t)\, \hat{A}_t\right]$$

- The family tree ran: A2C/A3C $\rightarrow$ +GAE $\rightarrow$ TRPO $\rightarrow$ **PPO** $\rightarrow$ **GRPO** (critic-free, group-mean baseline — the current LLM-RL default)
- **Key limitation (the one we attack today):** every method in that tree is *on-policy* — it *discards all experience after each update*. Reusing a replay buffer is the central win today
- *Secondary:* the actor is always *stochastic*. For a physical actuator you eventually just want one number per joint — no distribution to sample or integrate over

Recap: What Today Delivers

- **Today we go off-policy and continuous.**
- **DPG** [1]: a policy-gradient update rule for a *deterministic* actor $\mu_\theta(s)$ over continuous action spaces
- **DDPG** [2]: DPG made deep, by importing the DQN [3] tricks you already know — replay buffer and target networks
- The critic becomes an *action*-value $Q_\phi(s, a)$, and the actor's gradient flows *through the critic*: $\nabla_\theta Q_\phi(s, \mu_\theta(s))$
- We close with **TD3** [4], the direct fix for DDPG's overestimation and brittleness — and the bridge to Day 6 (SAC)

Scope note: PPO already handles continuous actions *on-policy*. "Continuous Control I/II" is specifically the *off-policy* branch: deterministic policies (DDPG/TD3, today) and max-entropy stochastic policies (SAC, Day 6).

Notation: What Carries Over, What Is New

Symbol	Meaning	Where you saw it
$\mathcal{J}(\theta)$	expected discounted return (the objective)	Days 3–4
θ	**actor** parameters	Days 3–4
ϕ	**critic** parameters	Day 4
$\theta^-,\ \phi^-$	*target* network parameters	Day 2 (DQN)
$\mathcal{D}$	replay buffer	Day 2 (DQN)
$\pi_\theta(a\|s)$	*stochastic* actor	Days 3–4
$\mu_\theta(s)$	*deterministic* actor (new today)	Day 3, as the Gaussian mean
$Q_\phi(s,a)$	*action*-value critic (new today)	Day 4's critic was $V_\phi(s)$

Why μ for the actor? Day 3 already wrote the continuous-action policy as a Gaussian, $\pi_\theta(a|s) = \mathcal{N}\big(\mu_\theta(s),\, \sigma_\theta(s)^2\big)$. Today we keep the mean and throw the noise away: the actor *is* $\mu_\theta(s)$. Same letter, same network — we have simply deleted σ_θ. Keeping π and μ distinct also matters on Day 6, where SAC puts a stochastic actor back.

Learning Outcomes

- Explain why you cannot take $\arg\max_a Q(s, a)$ over a continuous action space, and why that forces us to *learn* a deterministic actor μ_θ
- Explain why the Days 3–4 policy gradient stops working for a deterministic actor, and state the rule that replaces it: the critic supplies a *direction* $\nabla_a Q_\phi$ rather than a score
- Say what changes relative to Day 4's actor-critic: actor $\pi_\theta \to \mu_\theta$, critic $V_\phi(s) \to Q_\phi(s, a)$, on-policy rollouts $\to$ replay buffer
- Read the DDPG algorithm and locate the replay buffer, the target networks, the soft update, the exploration noise, and the two update rules
- Explain why DDPG's TD target needs no $\max_{a'}$, and what soft target updates and batch normalisation contribute to stability
- State DDPG's failure modes (Q-overestimation, brittleness, weak exploration) and how TD3's three fixes address them

What is *continuous control*?

Control: choosing actions over time to drive a dynamical system toward desired behaviour — exactly the RL problem on an MDP $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$ from Day 1.

- **Discrete control** (Days 1–2): the action set $\mathcal{A}$ is a *finite list* — move left/right, an Atari button, one of Go's 4,672 legal moves. The policy picks one entry.
- **Continuous control** (today): the action is a *real-valued vector* $a \in \mathbb{R}^d$, usually bounded, e.g. $\mathcal{A} = [-1, 1]^d$.
  - Robot arm: joint torques. Locomotion: motor commands. Driving: steering angle + throttle.
  - The set is *uncountable* — there is no list to enumerate.

What is *continuous control*?

Control: choosing actions over time to drive a dynamical system toward desired behaviour — exactly the RL problem on an MDP $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$ from Day 1.

- **Discrete control** (Days 1–2): the action set $\mathcal{A}$ is a *finite list* — move left/right, an Atari button, one of Go's 4,672 legal moves. The policy picks one entry.
- **Continuous control** (today): the action is a *real-valued vector* $a \in \mathbb{R}^d$, usually bounded, e.g. $\mathcal{A} = [-1, 1]^d$.
  - Robot arm: joint torques. Locomotion: motor commands. Driving: steering angle + throttle.
  - The set is *uncountable* — there is no list to enumerate.
- **Why this is not just a bigger discrete problem.** Go, Dota and StarCraft have *enormous but still discrete* action sets (10^{26} choices per step in StarCraft) — you can in principle enumerate them and take an arg max. In continuous control you cannot enumerate at all.

This lecture: how to do RL when actions are continuous.

DPG and DDPG in One Slide

We cannot search for $\arg\max_a Q(s,a)$ any more. So we stop searching, and **train a second network to output that action for us:** $\mu_\theta(s) \approx \arg\max_a Q_\phi(s,a)$. One forward pass replaces the search. That network is the **actor**.

DPG and DDPG in One Slide

We cannot search for $\arg\max_a Q(s,a)$ any more. So we stop searching, and **train a second network to output that action for us:** $\mu_\theta(s) \approx \arg\max_a Q_\phi(s,a)$. One forward pass replaces the search. That network is the **actor**.

DPG (Deterministic Policy Gradient) [1] — tells us how to *train* that actor

- Works over a *continuous* action space
- But it predates deep RL — linear function approximators only

DPG and DDPG in One Slide

We cannot search for $\arg\max_a Q(s,a)$ any more. So we stop searching, and **train a second network to output that action for us:** $\mu_\theta(s) \approx \arg\max_a Q_\phi(s,a)$. One forward pass replaces the search. That network is the **actor**.

DPG (Deterministic Policy Gradient) [1] — tells us how to *train* that actor

- Works over a *continuous* action space
- But it predates deep RL — linear function approximators only

DQN (Deep Q-Network) [3] — Day 2

- Learns value functions with *neural networks*, kept stable by a **replay buffer** and a **target network**
- But only over *discrete* action spaces (it needs that $\max_{a'}$)

DPG and DDPG in One Slide

We cannot search for $\arg\max_a Q(s,a)$ any more. So we stop searching, and **train a second network to output that action for us:** $\mu_\theta(s) \approx \arg\max_a Q_\phi(s,a)$. One forward pass replaces the search. That network is the **actor**.

DPG (Deterministic Policy Gradient) [1] — tells us how to *train* that actor

- Works over a *continuous* action space
- But it predates deep RL — linear function approximators only

DQN (Deep Q-Network) [3] — Day 2

- Learns value functions with *neural networks*, kept stable by a **replay buffer** and a **target network**
- But only over *discrete* action spaces (it needs that $\max_{a'}$)

DDPG [2] = DPG + DQN's tricks: a model-free, *off-policy*, actor-critic algorithm that learns a deterministic policy on a continuous action space.

Continuous Control: **Deterministic Policy Gradients**

The Day 3–4 Update Rule Stops Working

This is how we have trained every actor so far:

$$\nabla_\theta \mathcal{J}(\theta) \;=\; \mathbb{E}\Big[\, \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, \hat{A}_t \,\Big]$$

It works by changing **probabilities**: make good actions more likely, bad actions less likely.

The Day 3–4 Update Rule Stops Working

This is how we have trained every actor so far:

$$\nabla_\theta \mathcal{J}(\theta) \;=\; \mathbb{E}\Big[\, \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, \hat{A}_t \,\Big]$$

It works by changing **probabilities**: make good actions more likely, bad actions less likely.

But a deterministic actor has no probabilities.

- $\mu_\theta(s)$ returns *one* action. It is already certain — there is no "make it more likely".
- So $\log \pi_\theta(a_t|s_t)$ has nothing to compute, and the expectation has nothing to average over.

We need a different way to answer one question: **which way should I change θ?**

DPG: Let the Critic *Point* [1]

Feed the actor's action into the critic: $Q_\phi\big(s, \mu_\theta(s)\big)$. That number says how good the actor's choice was. **We want it bigger** — so climb it.

DPG: Let the Critic *Point* [1]

Feed the actor's action into the critic: $Q_\phi\big(s, \mu_\theta(s)\big)$. That number says how good the actor's choice was. **We want it bigger** — so climb it.

Two questions, one after the other:

1. Which way should the **action** move to raise the value? $\rightarrow \quad \nabla_a Q_\phi(s, a)$
2. Which way should the **weights** move to move the action that way? $\rightarrow \quad \nabla_\theta \mu_\theta(s)$

DPG: Let the Critic *Point* [1]

Feed the actor's action into the critic: $Q_\phi\big(s, \mu_\theta(s)\big)$. That number says how good the actor's choice was. **We want it bigger** — so climb it.

Two questions, one after the other:

1. Which way should the **action** move to raise the value? $\rightarrow \quad \nabla_a Q_\phi(s, a)$
2. Which way should the **weights** move to move the action that way? $\rightarrow \quad \nabla_\theta \mu_\theta(s)$

Multiply them. That is the **deterministic policy gradient**:

$$\nabla_\theta \mathcal{J}(\mu_\theta) \;=\; \mathbb{E}_{s\sim\mathcal{D}}\Big[\; \boxed{\nabla_\theta \mu_\theta(s)} \;\cdot\; \boxed{\nabla_a Q_\phi(s, a)\big|_{a=\mu_\theta(s)}} \;\Big]$$

In code this is one line: maximise $Q_\phi\big(s, \mu_\theta(s)\big)$ with ϕ held fixed. Autograd does both steps.

Old Rule vs. New Rule

Stochastic (Days 3–4):

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{s,\ a \sim \pi_\theta}[\nabla_\theta \log \pi_\theta(a \mid s)\ Q_\phi(s,a)]$$

Deterministic (today):

$$\nabla_\theta J(\mu_\theta) = \mathbb{E}_{s \sim \mathcal{D}}\Big[\nabla_\theta \mu_\theta(s)\ \nabla_a Q_\phi(s,a)\big|_{a=\mu_\theta(s)}\Big]$$

- The critic used to give a **score** — a number saying how hard to push. Now it gives a **direction** — which way to move the action.
- The old rule needs actions drawn from the *current* policy ($a \sim \pi_\theta$). **That is what forced it to be on-policy.**

This Is Still Day 4's Actor-Critic — Three Things Change

	Day 4 (A2C/PPO)	**Today (DDPG)**
Actor	$\pi_\theta(a\|s)$, stochastic	$\mu_\theta(s)$, deterministic
Critic	$V_\phi(s)$	$Q_\phi(s, a)$
Actor update	critic *scores*: $\nabla_\theta \log \pi_\theta$	critic *points*: $\nabla_a Q_\phi$
Data	on-policy rollout, then discard	replay buffer $\mathcal{D}$

Everything else is the loop you already know: two networks trained together, the critic by squared TD error, the actor from what the critic says.

Why the Critic Is Now $Q_\phi(s, a)$ Instead of $V_\phi(s)$

The new actor update asks the critic: *if I change the action, does the value go up?*

- $V_\phi(s)$ has no action in it. There is nothing to differentiate. It can say "this state is good", but not "turn the wheel left".
- So Day 4's trick of learning only V does not survive.

Why the Critic Is Now $Q_\phi(s, a)$ Instead of $V_\phi(s)$

The new actor update asks the critic: *if I change the action, does the value go up?*

- $V_\phi(s)$ has no action in it. There is nothing to differentiate. It can say "this state is good", but not "turn the wheel left".
- So Day 4's trick of learning only V does not survive.

The good news: Q costs us nothing extra. Building a TD target normally means finding the best next action — but the actor already hands us one:

$$y = r + \gamma\, Q_{\phi^-}\big(s',\, \boxed{\mu_{\theta^-}(s')}\big) \qquad \mathcal{L}(\phi) = \mathbb{E}\big[(y - Q_\phi(s,a))^2\big]$$

Same squared TD error as Days 2 and 4. Only the way we pick the next action has changed.

Background Summary

- **DPG** [1]: an update rule for *deterministic* actors, so we can learn a policy on a continuous action domain without ever computing $\arg\max_a Q$

  $\Rightarrow$ model-free, actor-critic

Background Summary

- **DPG** [1]: an update rule for *deterministic* actors, so we can learn a policy on a continuous action domain without ever computing $\arg\max_a Q$

  $\Rightarrow$ model-free, actor-critic

- **DQN** [3]: value functions with neural networks, made to converge by two tricks
  - **Target network** — a frozen copy supplies the TD target, so we are not chasing our own tail
  - **Replay buffer** — decorrelates consecutive samples $\Rightarrow$ off-policy

Background Summary

- **DPG** [1]: an update rule for *deterministic* actors, so we can learn a policy on a continuous action domain without ever computing $\arg\max_a Q$

  $\Rightarrow$ model-free, actor-critic

- **DQN** [3]: value functions with neural networks, made to converge by two tricks
  - **Target network** — a frozen copy supplies the TD target, so we are not chasing our own tail
  - **Replay buffer** — decorrelates consecutive samples $\Rightarrow$ off-policy

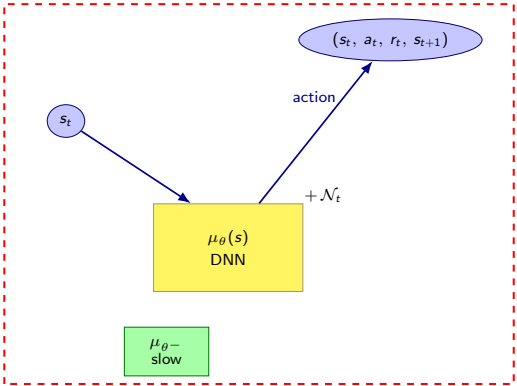
- **DDPG** [2]: learn *both* the actor and the critic of DPG with neural networks, using the DQN tricks to keep it stable

DDPG Problem Setting: Four Networks

$\mu_\theta(s)$	**Actor** network — deterministic, outputs an action in $\mathbb{R}^d$
$Q_\phi(s, a)$	**Critic** network — action-value
$\mu_{\theta^-}(s)$	**Target actor** — a slowly-following copy of μ_θ
$Q_{\phi^-}(s, a)$	**Target critic** — a slowly-following copy of Q_ϕ
$\mathcal{D}$	**Replay buffer** of transitions (s, a, r, s')

Target networks appear twice because the TD target $y = r + \gamma\, Q_{\phi^-}(s', \mu_{\theta^-}(s'))$ uses *both* of them — DQN only needed one, because it had no actor.

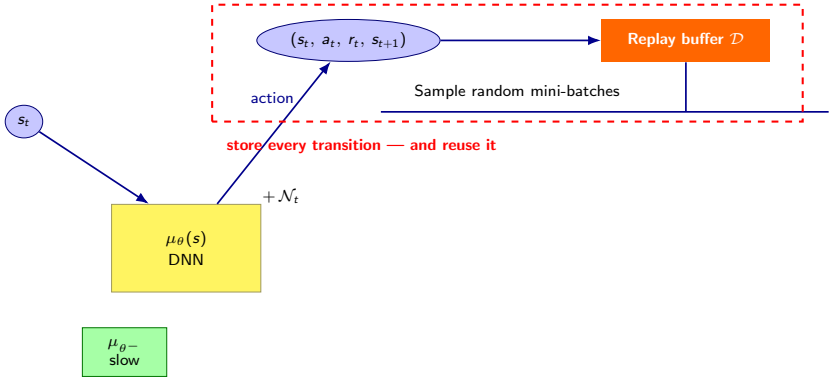
Method: The DDPG Loop (1/3) — Acting



act with the actor, + noise

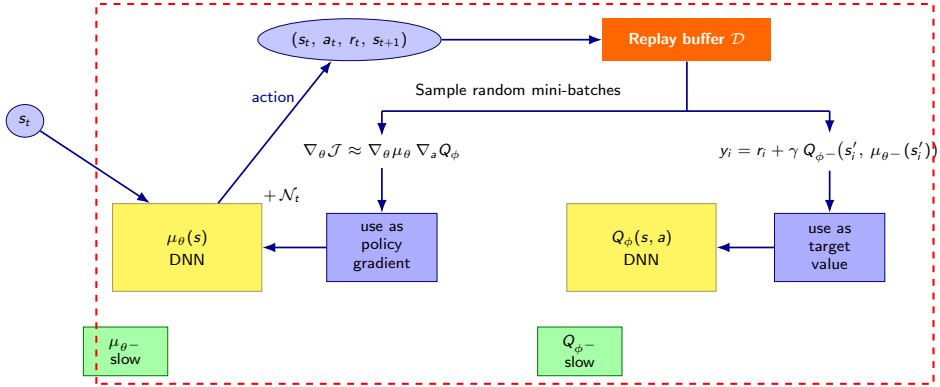
The actor is a plain function $s \mapsto \mu_\theta(s)$. Being deterministic it would repeat itself forever, so exploration noise $\mathcal{N}_t$ is added before acting. Each step yields one transition.

Method: The DDPG Loop (2/3) — Storing



Transitions go into the replay buffer $\mathcal{D}$ rather than being used once and discarded — the DQN trick from Day 2, and the reason DDPG needs far less environment interaction than PPO on the same task.

Method: The DDPG Loop (3/3) — Learning



One mini-batch drives *both* updates: on the right the "slow" target networks build the TD target y_i for the critic; on the left the critic's action-gradient becomes the actor's policy gradient. Nothing inside the dashed box asks which policy collected the data — that is what off-policy means.

Diagram adapted from [9]

The DDPG Algorithm

Initialise critic $Q_\phi(s, a)$ and actor $\mu_\theta(s)$ with random weights $\phi,\ \theta$
Initialise target weights $\phi^- \leftarrow \phi$ and $\theta^- \leftarrow \theta$
Initialise replay buffer $\mathcal{D}$
for episode $= 1$ **to** M **do**
  Observe initial state s
  for $t = 1$ **to** T **do**
    Select action $a = \mu_\theta(s) + \mathcal{N}$ *(actor output + exploration noise)*
    Execute a; observe reward r, next state s', terminal flag d
    Store transition (s, a, r, s', d) in $\mathcal{D}$; $s \leftarrow s'$
    Sample a minibatch of N transitions $(s_i, a_i, r_i, s_i', d_i) \sim \mathcal{D}$
    Set $y_i = r_i + \gamma\,(1 - d_i)\,Q_{\phi^-}\big(s_i',\ \mu_{\theta^-}(s_i')\big)$
    Update critic by descending $\nabla_\phi \frac{1}{N} \sum_i \big(y_i - Q_\phi(s_i, a_i)\big)^2$
    Update actor by ascending $\nabla_\theta \frac{1}{N} \sum_i Q_\phi\big(s_i,\ \mu_\theta(s_i)\big)$
    Soft-update targets: $\phi^- \leftarrow \tau\phi + (1-\tau)\phi^-;\ \ \theta^- \leftarrow \tau\theta + (1-\tau)\theta^-$
  end
end

Algorithm: DDPG — Deep Deterministic Policy Gradient

The DDPG Algorithm — The Two DQN Tricks

Replay buffer

Collect once, learn from it many times.

"Soft" target network update

Not DQN's hard copy every C steps.

Initialise critic $Q_\phi(s, a)$ and actor $\mu_\theta(s)$ with random weights $\phi,\ \theta$
Initialise target weights $\phi^- \leftarrow \phi$ and $\theta^- \leftarrow \theta$
Initialise replay buffer $\mathcal{D}$
for episode $= 1$ **to** M **do**
  Observe initial state s
  for $t = 1$ **to** T **do**
    Select action $a = \mu_\theta(s) + \mathcal{N}$ *(actor output + exploration noise)*
    Execute a; observe reward r, next state s', terminal flag d
    Store transition (s, a, r, s', d) in $\mathcal{D}$; $s \leftarrow s'$
    Sample a minibatch of N transitions $(s_i, a_i, r_i, s_i', d_i) \sim \mathcal{D}$
    Set $y_i = r_i + \gamma\,(1 - d_i)\,Q_{\phi^-}\big(s_i',\ \mu_{\theta^-}(s_i')\big)$
    Update critic by descending $\nabla_\phi \frac{1}{N} \sum_i \big(y_i - Q_\phi(s_i, a_i)\big)^2$
    Update actor by ascending $\nabla_\theta \frac{1}{N} \sum_i Q_\phi\big(s_i,\ \mu_\theta(s_i)\big)$
    Soft-update targets: $\phi^- \leftarrow \tau\phi + (1-\tau)\phi^-;\ \ \theta^- \leftarrow \tau\theta + (1-\tau)\theta^-$
  end
end

Algorithm: DDPG — Deep Deterministic Policy Gradient

Method — "Soft" Target Network Update

Day 2's DQN copies the online weights into the target network *periodically* — a hard update every C steps. DDPG instead lets the targets *slowly track* the online networks at every step:

$$\phi^- \leftarrow \tau\,\phi + (1-\tau)\,\phi^- \qquad\qquad \theta^- \leftarrow \tau\,\theta + (1-\tau)\,\theta^-$$

- $\tau \ll 1$ (typically $\tau = 10^{-3}$): the targets change very slowly — an exponential moving average of the online weights rather than a snapshot of them.

Method — "Soft" Target Network Update

Day 2's DQN copies the online weights into the target network *periodically* — a hard update every C steps. DDPG instead lets the targets *slowly track* the online networks at every step:

$$\phi^- \leftarrow \tau\,\phi + (1-\tau)\,\phi^- \qquad\qquad \theta^- \leftarrow \tau\,\theta + (1-\tau)\,\theta^-$$

- $\tau \ll 1$ (typically $\tau = 10^{-3}$): the targets change very slowly — an exponential moving average of the online weights rather than a snapshot of them.
- **The cost:** value information propagates backwards through time more slowly. This is the usual stability-versus-speed trade.

The DDPG Algorithm — Exploration

Add noise for exploration

A deterministic actor explores nothing on its own.

Initialise critic $Q_\phi(s, a)$ and actor $\mu_\theta(s)$ with random weights ϕ, θ
Initialise target weights $\phi^- \leftarrow \phi$ and $\theta^- \leftarrow \theta$
Initialise replay buffer $\mathcal{D}$
for episode $= 1$ **to** M **do**
  Observe initial state s
  for $t = 1$ **to** T **do**
    Select action $a = \mu_\theta(s) + \mathcal{N}$ *(actor output + exploration noise)*
    Execute a; observe reward r, next state s', terminal flag d
    Store transition (s, a, r, s', d) in $\mathcal{D}$; $s \leftarrow s'$
    Sample a minibatch of N transitions $(s_i, a_i, r_i, s_i', d_i) \sim \mathcal{D}$
    Set $y_i = r_i + \gamma\,(1 - d_i)\,Q_{\phi^-}\big(s_i',\ \mu_{\theta^-}(s_i')\big)$
    Update critic by descending $\nabla_\phi \frac{1}{N} \sum_i \big(y_i - Q_\phi(s_i, a_i)\big)^2$
    Update actor by ascending $\nabla_\theta \frac{1}{N} \sum_i Q_\phi\big(s_i,\ \mu_\theta(s_i)\big)$
    Soft-update targets: $\phi^- \leftarrow \tau\phi + (1-\tau)\phi^-$; $\theta^- \leftarrow \tau\theta + (1-\tau)\theta^-$
  end
end

Algorithm: DDPG — Deep Deterministic Policy Gradient

Method — Exploration with a Deterministic Actor

A deterministic actor has *no* exploration of its own: the same state always yields the same action, forever. On Day 3–4 the policy was stochastic, so exploration came for free. Here we must add it by hand:

$$a_t = \mu_\theta(s_t) + \mathcal{N}_t, \qquad \mathcal{N}_t \sim \mathcal{N}(0, \sigma^2)$$

- Zero-mean **Gaussian** noise added to the action, with σ annealed over training — the continuous-action analogue of annealing ε in ε-greedy (Day 1).

Method — Exploration with a Deterministic Actor

A deterministic actor has *no* exploration of its own: the same state always yields the same action, forever. On Day 3–4 the policy was stochastic, so exploration came for free. Here we must add it by hand:

$$a_t = \mu_\theta(s_t) + \mathcal{N}_t, \qquad \mathcal{N}_t \sim \mathcal{N}(0, \sigma^2)$$

- Zero-mean **Gaussian** noise added to the action, with σ annealed over training — the continuous-action analogue of annealing ε in ε-greedy (Day 1).
- **This is exploration only in the behaviour policy.** The stored actions include the noise; the actor being trained does not. Being off-policy is what allows that split.
- **It is also DDPG's weakest point.** The noise is undirected — it is a random jiggle, not a search. In sparse-reward tasks the agent may never stumble on a reward at all.

The DDPG Algorithm — Value Network Update

Value network update

Squared TD error — the same critic loss as Day 2 and Day 4.

Initialise critic $Q_\phi(s, a)$ and actor $\mu_\theta(s)$ with random weights $\phi,\ \theta$
Initialise target weights $\phi^- \leftarrow \phi$ and $\theta^- \leftarrow \theta$
Initialise replay buffer $\mathcal{D}$
for episode $= 1$ **to** M **do**
  Observe initial state s
  for $t = 1$ **to** T **do**
    Select action $a = \mu_\theta(s) + \mathcal{N}$ *(actor output + exploration noise)*
    Execute a; observe reward r, next state s', terminal flag d
    Store transition (s, a, r, s', d) in $\mathcal{D}$; $s \leftarrow s'$
    Sample a minibatch of N transitions $(s_i, a_i, r_i, s_i', d_i) \sim \mathcal{D}$
    Set $y_i = r_i + \gamma\,(1 - d_i)\,Q_{\phi^-}\big(s_i',\ \mu_{\theta^-}(s_i')\big)$
    Update critic by descending $\nabla_\phi \frac{1}{N} \sum_i \big(y_i - Q_\phi(s_i, a_i)\big)^2$
    Update actor by ascending $\nabla_\theta \frac{1}{N} \sum_i Q_\phi\big(s_i,\ \mu_\theta(s_i)\big)$
    Soft-update targets: $\phi^- \leftarrow \tau\phi + (1-\tau)\phi^-;\ \ \theta^- \leftarrow \tau\theta + (1-\tau)\theta^-$
  end
end

Algorithm: DDPG — Deep Deterministic Policy Gradient

The DDPG Algorithm — Policy Network Update

Initialise critic $Q_\phi(s, a)$ and actor $\mu_\theta(s)$ with random weights $\phi,\ \theta$
Initialise target weights $\phi^- \leftarrow \phi$ and $\theta^- \leftarrow \theta$
Initialise replay buffer $\mathcal{D}$
for episode $= 1$ **to** M **do**
  Observe initial state s
  for $t = 1$ **to** T **do**
    Select action $a = \mu_\theta(s) + \mathcal{N}$ *(actor output + exploration noise)*
    Execute a; observe reward r, next state s', terminal flag d
    Store transition (s, a, r, s', d) in $\mathcal{D}$; $s \leftarrow s'$
    Sample a minibatch of N transitions $(s_i, a_i, r_i, s_i', d_i) \sim \mathcal{D}$
    Set $y_i = r_i + \gamma\,(1 - d_i)\,Q_{\phi^-}\!\left(s_i',\ \mu_{\theta^-}(s_i')\right)$
    Update critic by descending $\nabla_\phi \frac{1}{N} \sum_i \left(y_i - Q_\phi(s_i, a_i)\right)^2$
    Update actor by ascending $\nabla_\theta \frac{1}{N} \sum_i Q_\phi\left(s_i,\ \mu_\theta(s_i)\right)$
    Soft-update targets: $\phi^- \leftarrow \tau\phi + (1-\tau)\phi^-;\ \ \theta^- \leftarrow \tau\theta + (1-\tau)\theta^-$
  end
end

Algorithm: DDPG — Deep Deterministic Policy Gradient

Policy network update

Climb the critic. This one line is the deterministic policy gradient.

In code the actor update is a single statement: maximise $\frac{1}{N} \sum_i Q_\phi(s_i, \mu_\theta(s_i))$ with ϕ held frozen. Autograd's chain rule then recovers the deterministic policy gradient, $\nabla_\theta \mu_\theta \cdot \nabla_a Q_\phi$.

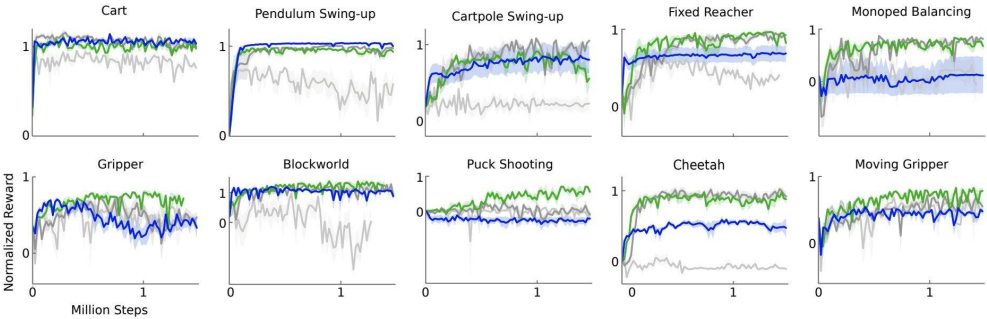
One Line Separates DDPG from DQN

Both algorithms are the same off-policy loop: act, store in $\mathcal{D}$, sample a minibatch, regress Q onto a bootstrapped target. They differ in *how the next action in the target is chosen*.

	TD target	**Next action from**
DQN (Day 2)	$y = r + \gamma \max_{a'} \hat{Q}(s', a'; \theta^-)$	a *search* over $\mathcal{A}$
DDPG (today)	$y = r + \gamma\, Q_{\phi^-}\big(s', \boxed{\mu_{\theta^-}(s')}\big)$	a *forward pass*

- **The target actor is the** arg max**.** That single substitution is what carries DQN into continuous action spaces — and it is why we needed the deterministic policy gradient: something has to *train* μ_θ to be that arg max.
- Everything else — replay buffer, target networks, squared TD error — is unchanged from Day 2.

Experiments



Light Grey: Original DPG

Dark Grey: Target Network

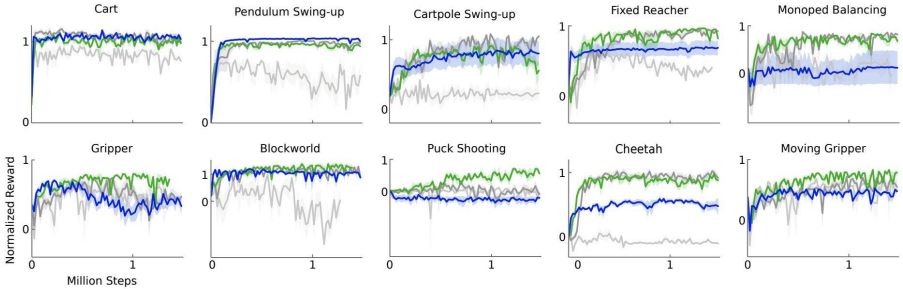
Green: Target Network + Batch Norm

Blue: Target Network from pixel-only

Figure: DDPG [2]

Experiments

Do target networks and batch norm matter?



Light Grey: Original DPG

Dark Grey: Target Network

Green: Target Network + Batch Norm

Blue: Target Network from pixel-only

Figure: DDPG [2]

Why batch normalisation here?

- Low-dimensional observations have components on *wildly different physical scales*: joint angles (radians), velocities, torques, positions (metres)
- A single network must learn across MuJoCo tasks where these ranges differ by orders of magnitude — raw inputs make the optimisation ill-conditioned
- **Batch normalisation** whitens each layer's activations (zero mean, unit variance per minibatch), so the same hyperparameters *transfer across tasks* with different state scales
- This is the green-vs-grey gap on the previous slide — batch norm is a genuine ablation variable, not a cosmetic add-on

Experiments — Is DDPG better than DPG?

Normalised return (1.0 ≈ planning baseline) across 30+ tasks.

DDPG columns: low-dim (*lowd*) and pixel (*pix*) inputs.

DPG columns: original control (*cntrl*).

environment	$R_{av,lowd}$	$R_{best,lowd}$	$R_{av,pix}$	$R_{best,pix}$	$R_{av,cntrl}$	$R_{best,cntrl}$
blockworld1	1.156	1.511	0.466	1.299	-0.080	1.260
blockworld3da	0.340	0.705	0.889	2.225	-0.139	0.658
canada	0.303	1.735	0.176	0.688	0.125	1.157
canada2d	0.400	0.978	-0.285	0.119	-0.045	0.701
cart	0.938	1.336	1.096	1.258	0.343	1.216
cartpole	0.844	1.115	0.482	1.138	0.244	0.755
cartpoleBalance	0.951	1.000	0.335	0.996	-0.468	0.528
cartpoleParallelDouble	0.549	0.900	0.188	0.323	0.197	0.572
cartpoleSerialDouble	0.272	0.719	0.195	0.642	0.143	0.701
cartpoleSerialTriple	0.736	0.946	0.412	0.427	0.583	0.942
cheetah	0.903	1.206	0.457	0.792	-0.008	0.425
fixedReacher	0.849	1.021	0.693	0.981	0.259	0.927
fixedReacherDouble	0.924	0.996	0.872	0.943	0.290	0.995
fixedReacherSingle	0.954	1.000	0.827	0.995	0.620	0.999
gripper	0.655	0.972	0.406	0.790	0.461	0.816
gripperRandom	0.618	0.937	0.082	0.791	0.557	0.808
hardCheetah	1.311	1.990	1.204	1.431	-0.031	1.411
hopper	0.676	0.936	0.112	0.924	0.078	0.917
hyq	0.416	0.722	0.234	0.672	0.198	0.618
movingGripper	0.474	0.936	0.480	0.644	0.416	0.805
pendulum	0.946	1.021	0.663	1.055	0.099	0.951
reacher	0.720	0.987	0.194	0.878	0.231	0.953
reacher3daFixedTarget	0.585	0.943	0.453	0.922	0.204	0.631
reacher3daRandomTarget	0.467	0.739	0.374	0.735	-0.046	0.158
reacherSingle	0.981	1.102	1.000	1.083	1.010	1.083
walker2d	0.705	1.573	0.944	1.476	0.393	1.397
torcs	-393.385	1840.036	-401.911	1876.284	-911.034	1961.600

Table: DDPG [2]

Experiments — DDPG still exhibits high variance

DDPG often reaches a strong *best* run, but its *average* return is far lower — a large gap signals instability across seeds/episodes:

Environment	$R_{av,\,lowd}$	$R_{best,\,lowd}$
cart	0.938	1.336
pendulum	0.946	1.021
canada	0.303	1.735
gripper	0.618	0.972
torcs	−393.4	1840.0

- Simple tasks (cart, pendulum): average ≈ best — stable.
- Harder tasks (canada, gripper) and especially **torcs**: average collapses while best stays high ⇒ **high variance / brittleness**.
- This overestimation-driven instability is exactly what **TD3** later targets.

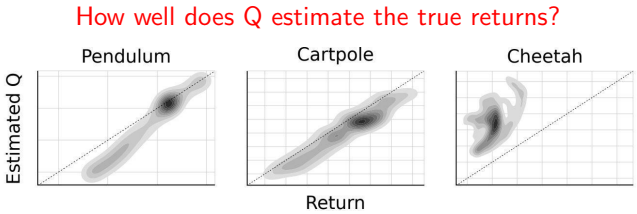


Figure 3: Density plot showing estimated Q values versus observed returns sampled from test episodes on 5 replicas. In simple domains such as pendulum and cartpole the Q values are quite accurate. In more complex tasks, the Q estimates are less accurate, but can still be used to learn competent policies. Dotted line indicates unity, units are arbitrary. (Figure: DDPG [2].)

TD3: DDPG Done Right [4]

The experiments showed DDPG *overestimates* Q and is brittle. **TD3** (Twin Delayed DDPG) keeps the deterministic off-policy actor-critic and adds three fixes:

1. **Clipped double-Q (twin critics):** learn two critics Q_{ϕ_1}, Q_{ϕ_2} and use the *smaller* of the two in the TD target, so the actor cannot exploit whichever one is optimistic:

$$y = r + \gamma \min_{i=1,2} Q_{\phi_i^-}(s', \tilde{a})$$

2. **Target policy smoothing:** $\tilde{a} = \mathrm{clip}\big(\mu_{\theta^-}(s') + \epsilon,\ a_{\min},\ a_{\max}\big)$ with $\epsilon \sim \mathcal{N}(0, \sigma)$ clipped to $[-c, c]$ — blurs the target so a sharp, spurious peak in Q cannot be chased.

3. **Delayed policy updates:** update the actor and the targets less often than the critics (e.g. once every 2 critic steps), so the actor climbs a critic that has had time to settle.

Takeaway: TD3 is the deterministic successor to DDPG, and its clipped double-Q idea carries directly into SAC (Day 6).

A Real Application: Wayve — "Learning to Drive in a Day" [6]

A full-size car learning lane-following on a real road, from scratch. All exploration and optimisation ran on-board; a simulator was used only to tune hyperparameters beforehand.

- **Task:** stay in lane on a real 250 m stretch of private road. **State:** a single monocular camera image. **Action:** *two continuous numbers* — steering angle in $[-1, 1]$ and a speed setpoint in km/h.
- **Reward:** forward speed. An episode ends when the safety driver intervenes, so a state's value is the distance driven before an intervention.
- **Result:** lane following learned in **11 episodes / 15 minutes** of real driving (35 episodes / 37 minutes when learning straight from raw pixels rather than a compressed image representation).
- **Why DDPG:** the authors deliberately used an *off-the-shelf* algorithm with no task-specific modification, to show the problem formulation was doing the work.
- **Why off-policy mattered:** those few hundred metres of driving were replayed many times over. An on-policy method would have needed far more driving than a safety driver can supply.

Continuous Control: **Summary**

The Algorithm Family Tree

DQN + DPG → **DDPG** → +clipped double-Q / delayed updates → **TD3** → +max-entropy → **SAC** (Day 6)

Algorithm	**On-policy?**	**Action space**	**Policy**	**Stability**
PPO / GRPO	Yes	Either	Stochastic π_θ	Good
DQN	No (replay)	Discrete	Implicit ($\arg\max Q$)	Medium
DPG	No	Continuous	Deterministic μ_θ	— (not deep)
DDPG	No (replay)	Continuous	Deterministic μ_θ	Brittle
TD3	No (replay)	Continuous	Deterministic μ_θ	Improved

Summary

- **DDPG = DPG + DQN.** Bypass the intractable $\arg\max_a Q$ by training a second network — the actor μ_θ — to output the maximising action
- **What changed from Day 4:** actor $\pi_\theta \to \mu_\theta$, critic $V_\phi(s) \to Q_\phi(s,a)$ (you cannot take ∇_a of a V), data on-policy $\to$ replay buffer
- **The deterministic policy gradient:** $\nabla_\theta J = \mathbb{E}_{s\sim\mathcal{D}}\left[\nabla_\theta \mu_\theta(s)\, \nabla_a Q_\phi(s,a)|_{a=\mu_\theta(s)}\right]$ — the critic *points* rather than weights
- Tricks that made it work:
  - **Replay buffer** & **target networks** (from DQN) — off-policy, stable TD targets
  - **Soft target updates** $\phi^- \leftarrow \tau\phi + (1-\tau)\phi^-$ instead of DQN's hard periodic copy
  - **Batch normalisation** — transfers across tasks with different state scales
  - **Added action noise** — the only exploration a deterministic actor has
- **TD3** fixes DDPG's Q-overestimation with clipped double-Q, target-policy smoothing, and delayed policy updates

Limitations of DDPG

- **Overestimation bias:** a single critic, maximised by the actor, systematically over-estimates Q (the experiments showed $R_{av} \ll R_{best}$)
- **Hyperparameter brittleness:** sensitive to learning rates, τ, noise scale, and architecture; results vary widely across seeds, and it scales poorly to harder tasks compared with TD3 and SAC
- **Deterministic exploration:** relies entirely on injected noise — weak and undirected in sparse-reward tasks ($\rightarrow$ Day 7: exploration)
- **Two-network instability:** the same coupled feedback loop Day 4 flagged, now worse — the critic *aims* the actor rather than just weighting it
- **Offline failure:** DDPG on a *fixed* buffer is essentially offline RL — it diverges from extrapolation error on out-of-distribution actions ($\rightarrow$ Day 8: motivates BCQ/CQL)

What's Next: Day 6

- DDPG and TD3 give us *off-policy, continuous control* — but with a **deterministic** policy and only weak, injected exploration

What's Next: Day 6

- DDPG and TD3 give us *off-policy, continuous control* — but with a **deterministic** policy and only weak, injected exploration
- **Day 6 — Soft Actor-Critic (SAC)** [5]**:** maximum-entropy RL
  - Stochastic policy with an explicit **entropy bonus**: $\mathcal{J}(\pi_\theta) = \mathbb{E}\big[\sum_t r_t + \alpha\,\mathcal{H}(\pi_\theta(\cdot|s_t))\big]$ — PPO's $c_2\mathcal{H}$ term promoted to a first-class citizen
  - Entropy drives *directed* exploration — no hand-tuned noise schedule
  - Reuses today's machinery: replay buffer, target networks, **TD3's clipped double-Q**
  - The deterministic policy gradient $\nabla_\theta Q_\phi(s, \mu_\theta(s))$ generalises to the **reparameterised** stochastic gradient — the noise returns, now differentiable
- **Day 7 — Exploration:** injected action noise is today's weakest part. *Hindsight Experience Replay* [7] — relabelling failed rollouts with the goal they *did* reach — is the standard companion to DDPG/TD3 in sparse-reward robotics
- Takeaway: SAC is the entropy-regularised, stochastic sibling of TD3 — today's machinery carries over

Continuous Control: **References**

References

[1] Silver, D., Lever, G., Heess, N., Degris, T., Wierstra, D., & Riedmiller, M. (2014).
Deterministic Policy Gradient Algorithms.
ICML 2014, PMLR 32, 387–395.

[2] Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., & Wierstra, D. (2015).
Continuous Control with Deep Reinforcement Learning.
arXiv:1509.02971; *ICLR* 2016.

[3] Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015).
Human-Level Control through Deep Reinforcement Learning.
Nature, 518(7540), 529–533.

[4] Fujimoto, S., van Hoof, H., & Meger, D. (2018).
Addressing Function Approximation Error in Actor-Critic Methods (TD3).
ICML 2018, PMLR 80, 1587–1596; arXiv:1802.09477.

[5] Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018).
Soft Actor-Critic: Off-Policy Maximum Entropy Deep RL with a Stochastic Actor.
ICML 2018, PMLR 80, 1861–1870; arXiv:1801.01290.

References (cont.)

[6] Kendall, A., Hawke, J., Janz, D., Mazur, P., Reda, D., Allen, J.-M., Lam, V.-D., Bewley, A., & Shah, A. (2018).
Learning to Drive in a Day.
arXiv:1807.00412; *ICRA* 2019, 8248–8254.

[7] Andrychowicz, M., Wolski, F., Ray, A., Schneider, J., Fong, R., Welinder, P., McGrew, B., Tobin, J., Abbeel, P., & Zaremba, W. (2017).
Hindsight Experience Replay (HER).
NeurIPS 2017; arXiv:1707.01495.

[8] Levine, S. CS 285: Deep Reinforcement Learning, UC Berkeley.
https://rail.eecs.berkeley.edu/deeprlcourse/

[9] Garg, A. CSC2621: Topics in Robotics — Reinforcement Learning (Winter 2020), University of Toronto.
https://www.pair.toronto.edu/csc2621-w20/

References

These slides have been adapted from

- Animesh Garg, CSC2621: Topics in Robotics — Reinforcement Learning, University of Toronto **[9]**
- Sergey Levine, CS 285: Deep Reinforcement Learning, UC Berkeley **[8]**